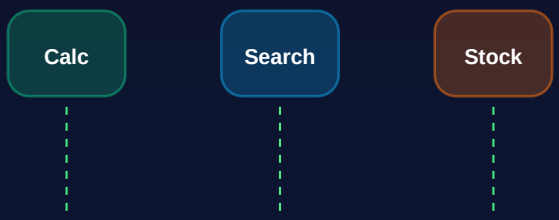


Tools in LangGraph

Agentic AI

Tool Nodes • Tools Condition • The Agent Loop

Detailed Lecture Notes with Full Code

CampusX Lecture Series

@tool Decorator | ToolNode | tools_condition | The Loop | bind_tools

■ TABLE OF CONTENTS ■

01	**Overview & Goal**	*From chatbot to Agentic AI*
02	**The Three Tools**	*Calculator, Search & Stock Price*
03	**Key LangGraph Concepts**	*ToolNode & tools_condition*
04	**The Agent Loop**	*Why linear is wrong, loops are right*
05	**Code: Defining Tools**	*@tool decorator & docstrings*
06	**Code: Binding & Graph**	*bind_tools + graph construction*
07	**Complete Working Code**	*Full implementation end-to-end*
08	**Frontend Integration**	*Streamlit filtering & status*
09	**Workflow Walkthrough**	*Step-by-step execution trace*
10	**Summary & Quick Reference**	*Everything at a glance*

01 Overview & Goal

This lecture focuses on upgrading a standard chatbot into an **Agentic AI** by adding **Tools**. An agent is an LLM that can not only generate text but also **decide when to take actions** in the real world — searching the web, doing math, calling APIs, and more.

Before (Standard Chatbot)	After (Agentic AI)
• Has memory & database • Can stream responses • **Limited to training data only** • **Cannot perform calculations** • **No real-time information**	• Everything from before + • **Can do math (Calculator)** • **Can search the internet** • **Can fetch live stock prices** • **Decides WHEN to use which tool**

■ The "Agent" Paradigm

What makes it "Agentic"? The key distinction is **autonomy**. The LLM doesn't just respond to text — it **analyzes the user's intent**, decides whether a tool is needed, selects the right tool, provides the correct inputs, interprets the output, and formulates a natural response. It is reasoning and acting in a loop.

02 The Three Tools Implemented

The tutorial implements three tools that showcase different categories of capabilities an agent can have:

■ Tool 1: Calculator

Type: Numerical Computation

Why Needed: LLMs are notoriously unreliable at math. They might say 47 × 83 = 3,891 (wrong!). A dedicated calculator tool guarantees correct results.

Operations: Add, Subtract, Multiply, Divide

Implementation: Custom function with `@tool` decorator

■ Tool 2: Internet Search (DuckDuckGo)

Type: Real-Time Information Retrieval

Why Needed: LLMs have a knowledge cutoff date. They cannot answer questions about current events, today's news, or recent developments without searching the web.

Source: DuckDuckGo (privacy-focused, no API key required)

Implementation: Prebuilt LangChain tool (`DuckDuckGoSearchRun`)

■ Tool 3: Stock Price Fetcher

Type: External API Integration

Why Needed: Fetches live financial data that changes every second. Demonstrates how agents can interact with any REST API.

Source: Alpha Vantage API (requires free API key)

Implementation: Custom function with `@tool` decorator + `requests` library

■ Key Pattern

The Pattern: Whether prebuilt or custom, every tool follows the same contract — it has a **name**, a **description** (docstring), **input parameters** with types, and returns **output data**. The LLM reads the description to decide when to use it.

03 Key LangGraph Concepts

LangGraph provides two critical prebuilt components that handle all the complexity of tool execution and routing:

■ ToolNode — The Tool Executor

A **prebuilt node** that acts as the "Tool Executor." You give it the list of tools, and it handles everything automatically:

- **Receives** tool call requests from the LLM (which tool + what inputs)
- **Dispatches** the request to the correct tool function
- **Executes** the function with the provided arguments
- **Captures** the output and packages it as a ToolMessage

```
from langgraph.prebuilt import ToolNode
```

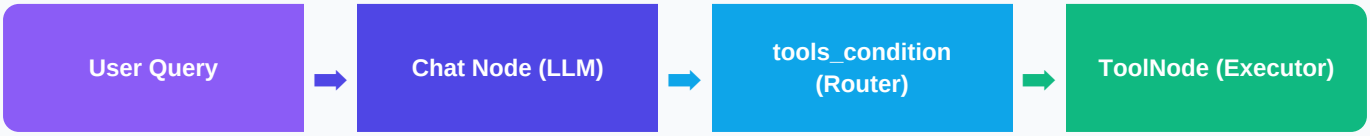
■ tools_condition — The Router

A **prebuilt conditional edge function** that acts as a decision-maker (router) after the Chat Node. It inspects the LLM's response:

- **Route A → END:** If the LLM generated a normal text response (no tool needed)
- **Route B → ToolNode:** If the LLM decided to call a tool (response contains tool_calls)

```
from langgraph.prebuilt import tools_condition
```

How They Fit Together:



04 The Agent Loop

This is the **most critical architectural decision** in building an agent. The video strongly emphasizes why a linear approach fails and why a loop is essential.

■ The Wrong Approach (Linear):

`START → Chat Node → Tool Node → END`

■ Why Linear Fails

Problem 1 — Raw Output: Tool outputs are often raw data (JSON from APIs, unformatted search results). Showing this directly to users is a terrible experience.

Problem 2 — No Multi-Step: What if the user asks "Get Apple's stock price AND calculate the cost of 50 shares"? A linear path ends after the first tool call, making multi-step reasoning impossible.

■ The Correct Approach (The Loop):

`START → Chat Node → Tool Node → Chat Node (back to LLM!)`

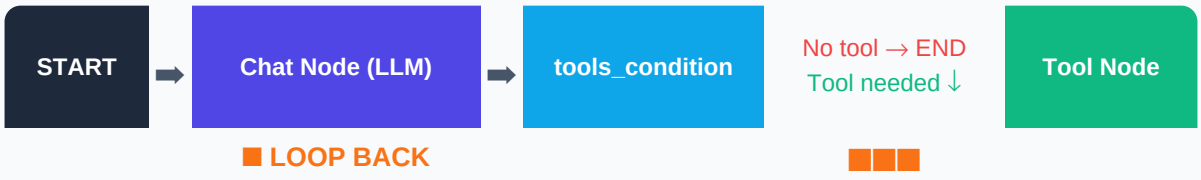
■ Why The Loop Works

Benefit 1 — Natural Responses: The LLM reads the raw tool output and generates a clean, human-readable answer. JSON becomes "Apple's stock price is $150."

Benefit 2 — Multi-Step Reasoning: After processing one tool's output, the LLM can decide to call *another* tool. The loop continues until the LLM has all the information it needs, then routes to END.

Benefit 3 — Self-Correction: If a tool returns an error, the LLM can retry with different inputs or try a different approach entirely.

The Agent Loop Visualized:



05 Code: Defining Tools

Tools in LangGraph are defined using LangChain's structure. There are two types: **Prebuilt Tools** (ready-made) and **Custom Tools** (you write them).

IMPORTS

```
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool
from langgraph.prebuilt import ToolNode, tools_condition
import requests
```

TOOL 1: INTERNET SEARCH (Prebuilt — one line!)

```
search_tool = DuckDuckGoSearchRun()
```

TOOL 2: CALCULATOR (Custom with @tool decorator)

```
@tool
def calculator(a: int, b: int, operation: str) -> int:
    """Performs basic arithmetic operations.
    Inputs:
    - a: first number
    - b: second number
    - operation: 'add', 'subtract', 'multiply', 'divide'
    """
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        return a / b
```

TOOL 3: STOCK PRICE FETCHER (Custom + API)

```
@tool
def get_stock_price(ticker: str):
    """Fetches the current stock price for a given
    company ticker symbol (e.g., AAPL, TSLA)."""
    url = f"https://www.alphavantage.co/query?"
          f"function=GLOBAL_QUOTE&symbol={ticker}"
          f"&apikey=YOUR_API_KEY"
    response = requests.get(url)
    return response.json()
```

✍ Crucial: Docstrings

Why Docstrings Are Critical: The LLM reads docstrings to understand what each tool does and when to use it. A poor docstring means the LLM will misuse or ignore the tool. Always include: what the tool does, what each parameter means, and example values.

06 Code: Binding & Graph

Step 1: Bind Tools to the LLM

Binding makes the LLM **aware** of the available tools. The LLM's response will now include `tool_calls` when it decides to use a tool.

```
# Collect all tools into a list
tools = [search_tool, calculator, get_stock_price]

# Bind tools to the LLM
llm_with_tools = llm.bind_tools(tools)
```

Step 2: Define the Chat Node

The chat node now uses `llm_with_tools` instead of the plain LLM.

```
def chat_node(state: State):
    # Uses tool-aware LLM
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}
```

Step 3: Build the Graph with the Loop

```
from langgraph.graph import StateGraph, START, END

# Initialize graph
builder = StateGraph(State)

# Add Nodes
builder.add_node("chat_node", chat_node)
builder.add_node("tool_node", ToolNode(tools))

# Add Edges
builder.add_edge(START, "chat_node")

# Conditional Edge: Router decides tools or END
builder.add_conditional_edges(
    "chat_node",
    tools_condition  # Prebuilt router
)

# ■ THE CRITICAL LOOP: tool output goes back to LLM
builder.add_edge("tool_node", "chat_node")

# Compile (with checkpointer for memory)
graph = builder.compile(checkpointer=memory)
```

■ Edge Types Summary

The 3 edge types used:

- **Normal edge:** `add_edge()` — unconditional flow (START→chat, tool→chat)
- **Conditional edge:** `add_conditional_edges()` — router decides (chat→tool OR chat→END)
- **Loop edge:** tool_node back to chat_node — creates the agent cycle

07 Complete Working Code

FULL BACKEND IMPLEMENTATION

```python
import os
from dotenv import load_dotenv
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import tool
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.memory import MemorySaver
import requests

load_dotenv()

# --- Define Tools ---
search_tool = DuckDuckGoSearchRun()

@tool
def calculator(a: int, b: int, operation: str) -> int:
    """Performs basic arithmetic."""
    ops = {"add": a+b, "subtract": a-b,
           "multiply": a*b, "divide": a/b}
    return ops.get(operation, "Invalid")

@tool
def get_stock_price(ticker: str):
    """Fetches current stock price for a ticker."""
    url = f"https://alphavantage.co/...{ticker}"
    return requests.get(url).json()

# --- Setup ---
tools = [search_tool, calculator, get_stock_price]
llm = ChatOpenAI(model="gpt-4").bind_tools(tools)
memory = MemorySaver()

# --- State & Node ---
class State(TypedDict):
    messages: Annotated[list, add_messages]

def chat_node(state):
    return {"messages": [llm.invoke(state["messages"])]}

# --- Build Graph ---
builder = StateGraph(State)
builder.add_node("chat_node", chat_node)
builder.add_node("tool_node", ToolNode(tools))
builder.add_edge(START, "chat_node")
builder.add_conditional_edges("chat_node", tools_condition)
builder.add_edge("tool_node", "chat_node")
graph = builder.compile(checkpointer=memory)
```

08 Frontend Integration

When integrating the tool-enabled backend with a Streamlit UI, two issues arise that need to be addressed:

■ Problem 1: Raw Tool Logs in Chat

When streaming responses, the graph emits **all** messages — including raw ToolMessages (system logs of API calls, JSON data, etc.). These look messy in the chat UI.

Solution: Filter the stream to only display `AIMessage` content.

FILTERING CODE

```
from langchain_core.messages import AIMessage

# Inside Streamlit streaming loop
for msg in graph.stream(inputs, config=config):
    # Only show AI responses, hide tool logs
    if isinstance(msg, AIMessage):
        st.write(msg.content)
```

■ Problem 2: No Feedback During Tool Use

While the agent is fetching data (which can take seconds), users see nothing happening. This creates a confusing "dead" period.

Solution: Use Streamlit's `st.status()` container to show "Using Calculator..." or "Searching the web..." indicators.

STATUS CONTAINER CODE

```
with st.status("Using tool...", expanded=True) as status:
    # Stream and process tool calls here
    status.update(label="Done!", state="complete")
```

09 Workflow Walkthrough

Let's trace through a complete example: **"What is the stock price of Apple?"**

1

■ User Query

User types: "What is the stock price of Apple?"

2

■ Chat Node Analyzes

LLM reads the query, recognizes it needs real-time data. Generates a tool_call: get_stock_price(ticker="AAPL")

3

■ tools_condition Routes

Router detects tool_calls in the response → Routes to Tool Node (not END)

4

■ Tool Node Executes

ToolNode dispatches to get_stock_price(). API call to Alpha Vantage. Returns raw JSON: {"Global Quote": {"05. price": "150.25"}}

5

■ Loop Back to Chat

Raw JSON is sent back to Chat Node as a ToolMessage in the conversation history

6

■ Chat Node Interprets

LLM reads the JSON and generates: "The current stock price of Apple (AAPL) is $150.25"

7

■ tools_condition Routes

Router detects NO tool_calls (just text) → Routes to END

8

■ Response Delivered

User sees: "The current stock price of Apple (AAPL) is $150.25" — clean and natural!

10 Summary & Quick Reference

Concept	Purpose	Key Code
@tool decorator	Marks a function as an LLM-usable tool	`@tool\ndef my_func():`
bind_tools()	Makes LLM aware of available tools	`llm.bind_tools(tools)`
ToolNode	Prebuilt node that executes tool calls	`ToolNode(tools)`
tools_condition	Router: tool_calls? → ToolNode, else → END	`add_conditional_edges(..., tools_condition)`
The Loop	Tool output goes back to LLM for processing	`add_edge("tool_node", "chat_node")`
Docstrings	LLM reads these to decide when to use a tool	`"""Detailed description..."""`

Calculator
Custom @tool Math operations

Search
Prebuilt tool DuckDuckGo API

Stock Price
Custom @tool Alpha Vantage API

The Big Takeaway

Tools transform a chatbot from a **text generator** into an **Agentic AI** that can reason, act, and interact with the real world. The secret sauce is the **Loop** — sending tool outputs back to the LLM so it can interpret raw data, perform multi-step reasoning, and generate clean, natural responses.

Define tools → Bind to LLM → Build graph with loop → Let the agent think.

Tools + bind_tools
Define & Register

+

ToolNode + Loop
Execute & Return

=

Agentic AI!
Reason + Act